作业 HW1* 实验报告

姓名：李欣洋 学号：2453359 日期：2025 年 9 月 26 日

1. 涉及数据结构和相关背景

数据结构：顺序表 链表
题目 1 涉及到链表和顺序表的常考算法：整体左移或者右移 k 个单位长度
题目 2 涉及到使用顺序表来对学生信息进行管理
题目 3 涉及到使用链表来存储多项式进行多项式的加减
题目 4 涉及到使用顺序表来对级数求和 其中涉及到高精度幂的运算
题目 5 涉及的是使用链表来实现一个扑克牌游戏

2. 实验内容

2.1 轮转数组

2.1.1 问题描述

给定一个整数顺序表 nums，将顺序表中的元素向右轮转 k 个位置，其中 k 是非负数。

2.1.2 基本要求

在输入时，要求能够输入两行数据，第一行输入两个整数 n 和 k，n 代表顺序表 nums 中的元素个数，k 表示要把顺序表中的元素右移 k 个位置。第二行的数据包括 n 个整数，代表顺序表中存储的 n 个元素。

2.1.3 数据结构设计

```
//定义了一个名为Node的结构体，用于维护顺序表以及顺序表的长度和可存储的数据规模
struct Node
{
    int* data;//指向该顺序表首地址的指针
    int length;//顺序表的长度
    int size;//顺序表初始化可存储的数据规模
    Node(int initSize)//Node的构造函数 initsize指定顺序表可存储的数据规模
    {
        data = new int[initSize];//在堆区开辟一段内存 用来作为顺序表 让data指向它
        length = 0;
        size = initSize;
    }
    ~Node()//Node的析构函数，在Node类型的对象结束时，释放堆区的内存
    {
        if(data!=nullptr)
        {
            delete[]data;
            data=nullptr;
        }
    }
};
```

2.1.4 功能说明（函数、类）

```
/*算法思路：双数组解决 开辟一个和数组data一样大小的数组temp，用来存储data中每一位数据
偏移之后的位置，然后将temp数组中的内容按相同位置赋值给data数组即可*/
int n,k;//n size
cin>>n>>k;
Node* node = new Node(n);//初始化顺序表
for(int i=0;i<n;i++)
{
    cin>>node->data[i];
    node->length++;//获取顺序表的长度
}
int* temp= new int[node->length];
for(int i=0;i<node->length;i++)
{
    temp[(i+k)%node->length]=node->data[i];//data中每个位置的数据偏移k个位置之后存储到temp中对应的位置
}
for(int i=0;i<node->length;i++)
{
    node->data[i]=temp[i];//把temp中的数据拷贝给data
}
```

2.1.5 调试分析（遇到的问题和解决方法）

此题在数据结构的设计上比较直观易懂，难点在于对算法的选取。
在双数组实现轮转数组算法的具体实现中没有遇到 warning 或者是 error，过程比较顺利。对此题的思考将写于 2.1.6 部分。

2.1.6 总结和体会

此题的难点我认为不是在于对数据结构的设计，而是在于对算法的选取。我个人选择的算法是双数组解法，而此解法的时间复杂度为 O(n)，空间复杂度为 O(n) （因为额外开辟了一个 temp 数组用来接受数据）。在解决此题目的过程中，我学会了一种处理轮转数组的优秀算法—三步翻转法。该算法先逆序数组，然后对前 k 个元素逆序，再对后 n-k 个元素逆序，就能够得到最终的数组。此种算法的时间复杂度也为 O(n)，但空间复杂度为 O(1)，因为不需要额外开辟新的数组。此算法在处理轮转数组问题中比较优秀。

2.2 学生信息管理

2.2.1 问题描述

定义一个包含学生信息（学号，姓名）的的 顺序表，使其具有如下功能：(1) 根据指定学生个数，逐个输入学生信息；(2) 给定一个学生信息，插入到表中指定的位置；(3) 删除指定位置的学生记录；(4) 分别根据姓名和学号进行查找，返回此学生的信息；(5) 统计表中学生个数。

2.2.2 基本要求

第一行输入一个数 n，代表顺序表中存储的学生个数。接下来 n 行输入学生的信息，学号和姓名之间用空格隔开。接下来执行若干行的命令，分别由 insert i number name（在第 i 个位置插入学号是 number 名字是 name 的学生，若 i 位置合法则输出 0，反之输出-1），remove j（删除第 j 个位置的学生，若 j 位置合法输出 0，反之输出-1），check no number（检查顺序表中是否存在学号为 number 的学生，若有则输出其位置，学号，姓名，反之输出-1），check name name（检查顺序表中是否存在名字为 name 的学生，若有则输出其位置，学号，姓名，反之输出-1）

2.2.3 数据结构设计

```
//定义一个名为Student的结构体，里面存储着no，name两个数据，no代表student的学号，name代表student的名字
struct Student
{
    string no;//学号
    string name;//名字
};
```

```
vector<Student> stu;//使用vector动态数组来存储 stu中的每一个元素是一个Student对象
for(int i=0; i<n; ++i)
{
    Student s;
    cin >> s.no>>s.name;
    stu.push_back(s);
}
```

2.2.4 功能说明（函数，类）

考虑到所有的命令是以字符串形式整行输入的，所以考虑使用 istringstream 函数对字符串进行分解（在遇到字符串中的空格自动断开，需要头文件 sstream），随后通过判断第一个命令（用 string 类型的变量 cmd 进行存储）的类型，根据 cmd 的类型不同来执行不同的操作

```
string input;
while(getline(cin,input) || input=="end")
{
    istringstream iss(input);//使用头文件sstream中的函数istringstream来分割输入的命令
    string cmd;
    iss>>cmd;
    if(cmd == "end") //如果读取到命令中的第一个指令是end 则执行end对应的操作
    if(cmd == "insert") //执行insert对应的操作
    else if(cmd == "remove")//执行remove对应的操作
    else if(cmd == "check") //执行check对应的操作
}
```

2.2.5 调试分析（遇到的问题和解决方法）

此题遇到的问题就是怎么选择合适的办法来读取以整串字符串形式输入的命令.起初的办法是考虑到每一行命令中的内容之间是用空格进行分割的,所以可以使用 cin 来进行读取,因为 cin 在遇到空格时会结束读取,这样的方法可行,但是稍显复杂,且在 cin 读取每一行时需要再最后清空缓冲区,避免缓冲区中有剩余的\n,从而导致后续的读取出现错误.所以思考之后考虑使用 getline 来对字符串进行整行读取,由此做到优化,然后使用 istringstream 来对读取的字符串进行分割,然后再把每一个不同的部分分别输入到对应的变量之中,由此就完成了对输入命令的处理!

2.2.6 总结和体会

在存储某一类数据时,比如存储学生的信息,该类数据通常拥有多个属性,比如学生的学号,姓名,性别,年龄等,此时我们可以先定义一个学生结构体,在该结构体中存储学号姓名等信息,然后把这个自定义数据类型作为元素存储到数组中.

2.3 一元多项式的相加和相乘

2.3.1 问题描述

一元多项式是有序线性表的典型应用，用一个长度为 m 且每个元素有两个数据项（系数项和指数项）的线性表((p1,e1),(p2,e2),...,(pm,em))可以唯一地表示一个多项式。 本题实现多项式的相加和相乘运算。本题输入保证是按照指数项递增有序的。

2.3.2 基本要求

输入:
第 1 行一个整数 m，表示第一个一元多项式的长度
第 2 行有 2m 个整数，p1 e1 p2 e2 ...，中间以空格分割,表示第 1 个多项式系数和指数
第 3 行一个整数 n,表示第二个一元多项式的项数
第 4 行有 2n 个整数，p1 e1 p2 e2 ...，中间以空格分割,表示第 2 个多项式系数和指数
第 5 行一个整数，若为 0,执行加法运算并输出结果，若为 1，执行乘法运算并输出结果；若为 2，输出一行加法结果和一行乘法的结果。
运算后的多项式链表，要求按指数从小到大排列
当运算结果为 0 0 时，不输出。

2.3.3 数据结构设计

考虑到一个单项式中的底数和指数是同时出现的关系,所以考虑使用了 pair 对来存储底数和指数,data.first 是底数,data.secon 是指数.
再使用链表作为数据结构,因为单项式是以升幂形式存储的,所以每一个单项式的 next 指针指向下一个指数比它大的单项式

```
/*定义一个名为Node的结构体，一个Node代表多项式中的一个单项式，其中pair对存储单项式的底数和指数，first是底数，
second是指数,next是指向下一个单项式的指针*/
struct Node
{
    pair<int,int>data;//存储单项式的底数和指数
    Node* next;//指向下一个单项式的指针
    Node(pair<int,int> val)//Node的构造函数
    {
        data=val;
        next=nullptr;
    };
};
```

2.3.4 功能说明（函数，类）

这是实现链表加法的函数

```
Node* Add(Node* l1,Node* l2)
{
    Node* p1=l1;
    Node* p,2=l2;
    Node* sum=new Node({0,0});
    Node* temp=sum;
    //当 p1 和 p2 均不为 nullptr 时,进入下列循环
    while(p1 && p2)
    {
        //当 p1 的指数比 p2 的指数大时,p1 不变,temp 指向 p2,p2 更新
    为 p2->next
        if(p1->data.second>p2->data.second)
```

```
    //当 p1 的指数比 p2 的指数小时,p2 不变,temp 指向 p1,p1 更新
    为 p1->next
        else if(p1->data.second<p2->data.second)
   //当 p1 和 p2 指数相等时,把 p1 和 p2 的底数相加,指数不变,随后 p1 和 p2 均更新
为 p1->next 和 p2->next
        else
    }
    //说明 p2 已经遍历结束了,把 p1 剩余部分按顺序接到 temp 的后面即可
    while(p1)
    //说明 p1 已经遍历结束了,把 p2 剩余部分按顺序接到 temp 的后面即可
    while(p2)
    //sum 是一个哑节点 不存储实际的数据,所以返回 sum 节点的 next
    return sum->next;
}
这是实现链表乘法的函数
Node* mul(Node* l1,Node* l2)
{
    //算法思路:多项式的乘法需要每一个单项式去乘另一个多项式,如果采取简单的遍
历,存储数据是一个很困难的事情,但是我们考虑到一个多项式中,指数只会出现一次(因为
合并同类项),这恰好与 set 或者 map 容器中 key 值的性质类似,由此考虑使用 set 或者
map 容器来存储,但是 set 中的 key 值只允许出现一次,所以考虑使用 map
    map<int,int>terms;//terms 中的一个 pair 对代表一个单项式
    for(Node* p1=l1;p1!=nullptr;p1=p1->next)
    {
        for(Node* p2=l2;p2!=nullptr;p2=p2->next)
        {
//底数相乘
            int coef = p1->data.first * p2->data.first;
//指数相加
            int exp = p1->data.second + p2->data.second;
//这里是关键,用指数做 key 值,把 value 的值作为底数,每次相乘遇到相同的指数(即同
类项),就把底数相加
            terms[exp] += coef;
        }
    }
    Node* product=new Node({0,0});
    Node* temp=product;
    //使用 map 容器的迭代器,用 temp 指向单项式
    for(auto it=terms.begin();it!=terms.end();it++)
    {
        temp->next=new Node(*it);
        temp=temp->next;
    }//product 一样是哑节点,因此返回 product->next
    return product->next;
```

```
}
```

2.3.5 调试分析（遇到的问题和解决方法）

一开始解决这道题目,想了很多的方法但是无法完成多项式的乘法,一开始考虑的是直接创建一个新链表的头节点,然后开始遍历进行两个链表的相乘,每一个乘积都去新链表中遍历一次,若指数存在,则底数相加,若不存在,则由小到大排序插入到合适的位置,但是这样的做法时间复杂度过大,是 O(n*m*p)(这里的 p 是新链表的长度).

因此考虑优化,考虑到了多项式中的指数只会出现一次的缘故,联想到了 set 和 map 容器中的 key 值也具有唯一性,又考虑到 map 中的 key 值允许出现多次,并且可以把 value 的值进行相加,所以考虑到使用 map 容器来进行存储.

使用此优化过后的算法,能够把时间复杂度降低至 O(n*m).

2.3.6 总结和体会

在这道题目中,算法思维得到了进一步的提升.其中在处理乘法时我认为使用了一些分治算法的思想,把乘法拆解为各项的结果之后再把相同的结果进行相加

在解决问题的过程中,也加深了我对 STL 的理解,在处理问题过程中,我们可以去关注某些数据的特性,比如说仅会出现一次的数据,我们可以把它和 set,map 中的 key 值联系起来,加强对 STL 的理解,能够使代码变得更加简洁和优雅.

2.4 求级数

2.4.1 问题描述

求解 $\sum_{i=1}^{N} iA^i$

2.4.2 基本要求

若干行，在每一行中给出整数 N 和 A 的值，（1<=N<=150，0<=A<=15）
对于 20%的数据，有 1<=N<=12，0<=A<=5
对于 40%的数据，有 1<=N<=18，0<=A<=9
对于 100%的数据，有 1<=N<=150，0<=A<=15

2.4.3 数据结构设计

```
/*定义了一个Series类，类中存储了一个vector容器，用来动态存储一个数字的每一位
理由：常规的for循环解决级数和，在遍历次数过多的时候，其结果会超出long long型的范围
因此考虑使用数组来完成高精度的存储*/
class Series
{
    private:
    //使用vector来存储数字的每一位 series[0]代表个位 由0增加代表由个位变成十位 百位.....
    vector<int> series;
```

```
Series result(0);
Series power(1);
for(int i=1;i<=N;i++)
{
    power=power*A;
    result=result+power*(i);
}
```

因为考虑到当指数过大时会发生指数爆炸的现象,而此时的数据大小已经完全超出了 longlong 型的范围,所以联系到高程中学习过的高精度幂的运算,我们考虑使用数组来完成对数据的存储,又因为数据的位数是动态的,所以使用 vector 容器来解决问题.

2.4.4 功能说明（函数，类）

以下是三个核心函数的部分代码,只选取了重要部分

```
//overload +
//重载+运算符,因为Series 是一个自定义类型,无法直接进行+操作
    Series operator+(const Series& other)const
    {
        for(int i=0;i<n;i++)
        {
            int digit1=i<series.size()?series[i]:0;
            int digit2=i<other.series.size()?other.series[i]:0;
//相同位置即相同的位,相加起来再加上上一位的进位
            int sum=digit1+digit2+carry;
            result.series.push_back(sum%10);
            carry=sum/10;
        }
    }
    //overload *
    //重载*运算符,也是因为Series 自定义数据类型无法直接进行*
    Series operator*(const int n)const
    {
        for(int i=0;i<series.size();i++)
        {
//每一位都与 n 进行相乘,并且计算余数
            int num=series[i]*n+carry;
            result.series.push_back(num%10);
            carry=num/10;
        }
    }
```

```
    //overload <<
    //重载<<进行输出,定义对Series 的输出方式
    friend ostream& operator<<(ostream& output,Series& s)
    {
//这里的遍历是从 s 的末尾开始的,这是因为 s 数组由低到高是从个位开始,这和我们书写数字的习惯恰好相反,我们习惯先书写最高位,而最高位是数组中的最后一位,所以要到序进行输出
       for(int i = s.series.size() - 1; i >= 0; i--)
       {
         output << s.series[i];
       }
    return output;
    }
```

2.4.5 调试分析（遇到的问题和解决方法）

在初次完成代码之后,我使用测试案例进行输出,发现结果都是错误的,但是我观察错误之后,发现我输出的结果和实际正确的结果刚好是到序的
我使用的代码如下

```
for(auto it=s.begin();i!=s.end();i++)
{
    output<<s.eries[i];
}
```

这时我就突然发现,我的存储方式是从个位开始的,所以我的输出应该从高位输出到低位,意识到这个问题之后,我就想把迭代器换为 end 递减到 begin,但是此时又出现了一个新的问题,end 迭代器指向的是 vector 容器中最后一位元素的下一位,所以如果访问 end 迭代器会出现越界行为,所以我选择了使用 size 函数获取 vector 的元素个数,然后进行反向遍历,就解决了这样的问题.

2.4.6 总结和体会

在处理幂运算等会导致数据范围过大时的问题,我们可以考虑使用顺序表存储数据的每一位来解决范围溢出的问题.使用这个方法可以解决很多高精度的问题.

同时在操作自定义对象时,如果有需要进行加减乘除的操作,选择重载运算符是一个很好的选择.

2.5 扑克牌游戏

2.5.1 问题描述

扑克牌有 4 种花色：黑桃（Spade）、红心（Heart）、梅花（Club）、方块（Diamond）。每种花色有 13 张牌，编号从小到大为：A,2,3,4,5,6,7,8,9,10,J,Q,K。
对于一个扑克牌堆，定义以下 4 种操作命令：
1) 添加（Append）：添加一张扑克牌到牌堆的底部。如命令“Append Club Q”表示添加一张梅花 Q 到牌堆的底部。

2) 抽取（Extract）：从牌堆中抽取某种花色的所有牌，按照编号从小到大进行排序，并放到牌堆的顶部。如命令“Extract Heart”表示抽取所有红心牌，排序之后放到牌堆的顶部。
3)反转（Revert）：使整个牌堆逆序。
4)弹出（Pop）：如果牌堆非空，则除去牌堆顶部的第一张牌，并打印该牌的花色和数字；如果牌堆为空，则打印 NULL。
初始时牌堆为空。输入 n 个操作命令（1 ≤ n ≤200），执行对应指令。所有指令执行完毕后打印牌堆中所有牌花色和数字（从牌堆顶到牌堆底），如果牌堆为空，则打印 NULL
注意：每种花色和编号的牌数量不限。

2.5.2 基本要求

第一行输入一个整数 n，表示命令的数量。
接下来的 n 行，每一行输入一个命令。
输出若干行，每次收到 Pop 指令后输出一行（花色和数子或 NULL），最后将牌堆中的牌从牌堆顶到牌堆底逐一输出（花色和数字），若牌堆为空则输出 NULL

2.5.3 数据结构设计

```
//卡牌类 用来表示单张卡牌 其中存储的三个成员分别代表 排的大小（rank） 牌的种类（suit） 指向下一张牌的指针（next）
class Card
{
    public:
    string rank;//牌的大小
    string suit;//牌的种类
    Card* next;//指向下一张牌的指针
    //Card的构造函数
    Card(string s,string r):suit(s),rank(r),next(nullptr){};
    Card(){};
};
```

2.5.4 功能说明（函数，类）

```
/*Desk 类 代表操作对卡牌的操作 其中private 中的成员变量head 代表牌堆的顶部 其中有Append Extract Pop Revert 四个函数 还有一个gethead 函数用来获取牌堆的顶部 便于操作*/
class Desk
{
    private:
    Card* head;//牌堆的顶部
    int getRankValue(string rank) //把AJQK 四个牌的点数改写为数字

    public:
    //Append 完成对牌的插入 suit rank 分别代表插入牌堆底的牌的种类和大小
    void Append(string suit,string rank)

    //Extract 完成对牌的抽取 s 代表要抽取的牌的种类
```

```
    void Extract(string s)
    {
        //用vector 来存储符合suit 种类的牌的节点，因为考虑到使用容器的话能够使用STL 中封装好的sort 函数来进行排序
        vector<Card*> seen;

        //sort
        sort(seen.begin(), seen.end(), [this](Card* a, Card* b)
        {
            return getRankValue(a->rank) <= getRankValue(b->rank);
        });
        //使用反向迭代器 因为题目要求由小到大插入到牌堆顶部 rebegin 迭代器指向seen 中rank 值最大的一个元素
        for (auto it = seen.rbegin(); it != seen.rend(); ++it)
        {
            (*it)->next = head;
            head = *it;
        }
    }
    //Pop 删除牌堆顶部的牌 无需参数传入
    void Pop()

    //Revert 反转牌堆 无需参数传入
    void Revert()
    {
        Card* prev=nullptr;
        Card* cur=head;
        //双指针法来完成链表的反转 cur 指向prev 每一次移动 cur 都重新赋值为cur->next
        while(cur)
        {
            Card* next=cur->next;
            cur->next=prev;
            prev=cur;
            cur=next;
        }
        head=prev;
    }
    //gethead 获取牌堆的顶部
    Card* gethead()
    {
        return head;
    }
};
```

2.5.5 调试分析（遇到的问题和解决方法）

最开始我将 卡牌的信息存储以及对卡牌的操作函数写在了一个 Card 类之中,但是在后续的代码书写时产生了很多的问题,比如说指针的维护,难度太大,而且在出现错误时,第一时间很难去定位是卡牌的存储出现了错误还是对卡牌的操作函数出现了错误.

所以我选择了把卡牌的信息单独作为了一个 Card 类,然后写了一个叫 Desk 的类来封装对卡牌的操作函数,这样可以让每个类只关注自己的职责,而不会产生相互的影响,也算是把面向对象的内容和数据结构的内容灵活的结合在了一起,使得代码方便维护也具有更强的拓展性.

2.5.6 总结和体会

通过这段卡牌管理程序的编写，我体会到面向对象设计的强大之处。将 Card 类与 Desk 类分离，实现了数据与操作的完美解耦，使代码结构清晰、职责分明。Card 类专注存储单张牌信息，Desk 类负责整体操作逻辑，这种设计不仅提高了代码的可读性和可维护性，更便于功能扩展和单元测试。在 Extract 方法中巧妙运用 STL 算法和反向迭代器，还灵活的使用了 STL.这样的经历能够积累很多代码经验.

2. 实验总结

通过本次数据结构实验，我系统掌握了顺序表和链表在不同场景下的应用。从轮转数组的算法优化、学生信息管理的模块化设计，到多项式运算的 STL 巧妙运用、高精度计算的运算符重载，再到扑克牌游戏的面向对象架构，每个实验都深化了我对数据结构核心思想的理解。我初步理解到到优秀代码不仅需要正确的算法，更需要清晰的结构设计——设计数据结构,善用 STL,设计不同的模块类等工程化思维至关重要。这些实践锻炼了我将理论知识转化为解决实际问题的能力，为后续的计算机学习巩固了基础。

在未来的数据结构实验中,我还将持续学习不同的算法以及对应的使用场景,同时学习一些在实际开发中常用的模范 code style!